

CodeViz — Architecture & Design Document

Product: Visual DSA learning platform
Repository: stealth-startup
Version: August 2026
Document type: High-Level Design (HLD) + Low-Level Design (LLD)

Table of Contents

- 1. [Executive Summary](#)
 - 2. [High-Level Design](#)
 - 3. [End-to-End Code Flows](#)
 - 4. [Database Schema](#)
 - 5. [API Reference](#)
 - 6. [Frontend \(LLD\)](#)
 - 7. [Backend \(LLD\)](#)
 - 8. [Electron, Docker & CI](#)
 - 9. [AlgoViz iOS \(Separate Product\)](#)
 - 10. [Technology Stack](#)
-

1. Executive Summary

CodeViz is a cross-platform data structures and algorithms (DSA) learning platform. Users write real code in seven languages, execute it on the server, and step through execution with:

- Variable diffs at each step
- Array bar charts
- 2D matrix grids
- Binary tree views (Java `TreeNode`)

The system ships in two forms:

Deployment	How it runs
Desktop	macOS Electron app — local API + embedded UI
Web beta	Single Docker container on Render (<code>codeviz-beta.onrender.com</code>)

A separate **AlgoViz** iOS app provides curated SwiftUI animations and is **not** connected to the CodeViz backend.

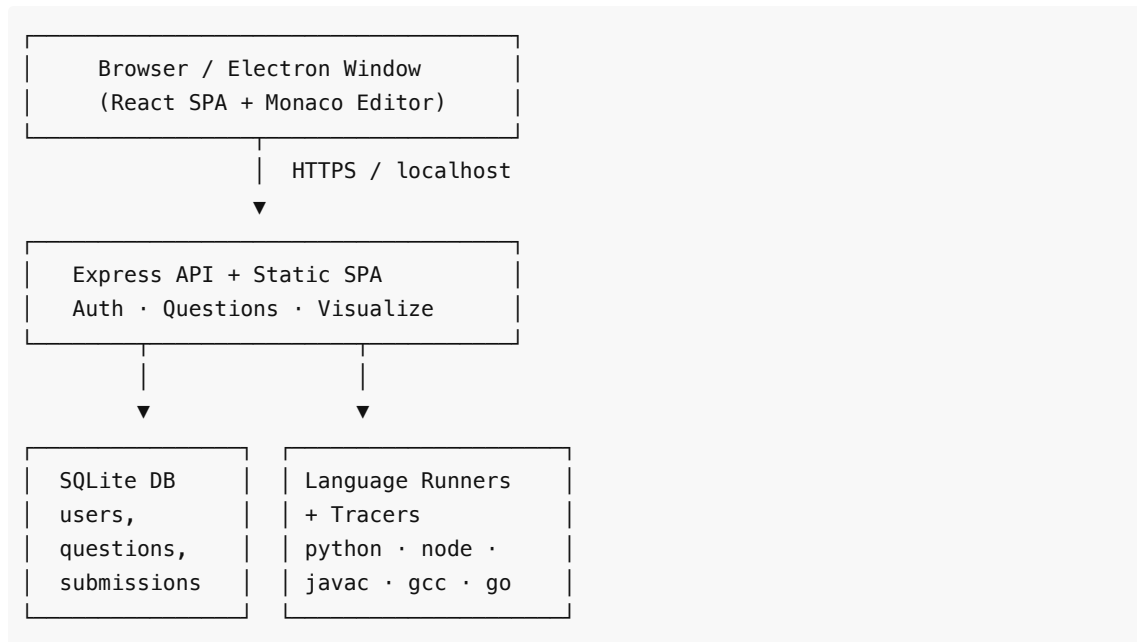
Key capabilities

- User authentication with JWT and per-user private question libraries
 - Monaco code editor with per-question language switching
 - Step-by-step visualization via language-specific code tracers
 - Optional Stripe billing (\$2/month, 7-day trial — disabled by default)
 - Support for Python, JavaScript, Java, C, C++, C#, and Go
-

2. High-Level Design (HLD)

2.1 System context

CodeViz uses a classic three-tier architecture:



The visualization engine runs **inside the backend**, using native compilers and interpreters installed on the host (or inside the Docker image).

2.2 Major components

Component	Location	Responsibility
Frontend	desktop/src/	React SPA, Monaco editor, visualization player
API Server	desktop/server/	Express routes: auth, billing, questions, visualize
Runners	desktop/server/runners/	Execute and instrument user code per language
Tracers	desktop/server/*-tracer/	Inject trace calls; capture steps at runtime
Electron Shell	desktop/electron/	Desktop packaging; spawns API on port 3847
Web Entry	desktop/server/standalone.ts	Single-port deploy for Docker / Render
Shared Types	desktop/shared/types.ts	TypeScript contracts between frontend and backend

2.3 Deployment topology

Desktop (Electron)

1. Electron main process starts Express on **port 3847**
2. Opens `BrowserWindow` loading:
 - Vite dev server (`:5173`) in development
 - `dist/index.html` in production

3. User data and SQLite live in the OS `userData` folder

Web (Docker / Render)

1. `standalone.ts` binds `0.0.0.0:PORT` (default 10000)
 2. Serves `dist/` SPA and `/api` routes from one process
 3. Docker image includes Python, GCC, JDK, Go (C# omitted for image size)
 4. Data persisted at `CODEVIZ_DATA_DIR` (`/app/data` on Render)
 5. Blueprint defined in `render.yaml` → `codeviz-beta` service
-

3. End-to-End Code Flows

3.1 Authentication

```
LoginPage
→ api.login()
→ POST /api/auth/login
    → bcrypt.compare(password)
    → JWT sign → { token, user }
→ localStorage: codeviz_token
→ AuthContext.refreshUser()
→ GET /api/billing/status
→ All later requests: Authorization: Bearer <token>
→ authMiddleware: verify JWT, load user from SQLite
```

3.2 Question CRUD

```
MyQuestionsPage
→ api.createQuestion()
→ POST /api/questions
    → Zod validation
    → createQuestionForUser()
    → INSERT questions + starter_code + solution_code

DashboardPage
→ api.getQuestions()
→ GET /api/questions (owner-scoped list)

EditorPage
→ api.getQuestion(id)
→ loads starter/solution code per language
```

3.3 Visualization (core flow)

This is the heart of CodeViz:

```
EditorPage – user clicks "Visualize code"
→ api.visualize({ language, code, questionId })
→ POST /api/visualize
    → authMiddleware
    → subscriptionMiddleware (if billing enabled)
```

- switch(language) → appropriate runner
- [Instrument source code]
- [Compile / Run]
- [Parse __CODEVIZ_JSON__ from stdout]
- INSERT submissions (audit log)
- return VisualizationResult { steps, finalOutput, success }

VisualizationPlayer (frontend)

- scrub through steps
- highlight current line in Monaco
- render arrays / matrices / trees from step.variables

3.4 Tracer pipeline (per language)

Step	What happens
1	User code received by /api/visualize
2	Runner selects instrumenter (e.g. instrumentJava)
3	Instrumenter injects trace calls at completed statements
4	Runtime tracer (Trace.java, trace.c, etc.) records variables per step
5	On exit, tracer emits __CODEVIZ_JSON__ { steps, finalOutput, success }
6	Runner parses JSON → VisualizationStep[] returned to frontend

3.5 Billing (optional)

Billing is **OFF** by default (BILLING_ENABLED=false).

When enabled:

Register → trial_ends_at = now + 7 days, status = trial
 Trial expires → subscriptionMiddleware returns 402 on /api/visualize
 BillingPage → POST /api/billing/checkout → Stripe Checkout (\$2/mo)
 Stripe webhook → updates subscription_status, stripe IDs in users table

4. Database Schema

Engine: node:sqlite (DatabaseSync)

Path: {CODEVIZ_DATA_DIR}/codeviz.db

Mode: WAL

users

Column	Type	Notes
id	TEXT	Primary key
email	TEXT	Unique

password_hash	TEXT	bcrypt
role	TEXT	user or admin
trial_ends_at	TEXT	ISO timestamp
subscription_status	TEXT	trial, active, past_due, cancelled, expired
stripe_customer_id	TEXT	Optional
stripe_subscription_id	TEXT	Optional
created_at	TEXT	ISO timestamp

questions

Column	Type	Notes
id	TEXT	Primary key
title	TEXT	
description	TEXT	Min 10 characters
difficulty	TEXT	beginner, intermediate, advanced
created_by	TEXT	FK → users.id
created_at	TEXT	

question_starter_code / question_solution_code

Column	Type	Notes
question_id	TEXT	FK → questions.id
language	TEXT	One of 7 supported languages
starter_code / solution_code	TEXT	Source code

Composite primary key: (question_id, language)

submissions

Column	Type	Notes
id	TEXT	Primary key
user_id	TEXT	FK → users.id
question_id	TEXT	FK → questions.id
language	TEXT	
code	TEXT	Submitted source
created_at	TEXT	Audit log of visualize runs

5. API Reference

Method	Endpoint	Description
GET	/api/health	Health check (Render)
POST	/api/auth/register	Create account + sample question
POST	/api/auth/login	Login, returns JWT
GET	/api/questions	List current user's questions
GET	/api/questions/:id	Get question with all language code
POST	/api/questions	Create question
PUT	/api/questions/:id	Update question
DELETE	/api/questions/:id	Delete question
POST	/api/visualize	Run code, return trace steps
GET	/api/billing/status	User + billing state
POST	/api/billing/checkout	Stripe checkout URL
POST	/api/billing/portal	Stripe customer portal
POST	/api/billing/webhook	Stripe webhook events

6. Frontend (Low-Level Design)

6.1 Routing (App.tsx)

Path	Page	Access
/login	LoginPage	Public
/register	RegisterPage	Public
/dashboard	DashboardPage	Protected
/questions	MyQuestionsPage	Protected
/editor/:id	EditorPage	Protected
/billing	BillingPage	Protected

All protected routes are wrapped in `ProtectedLayout` (auth gate + sidebar shell).

6.2 Key files

File	Purpose
desktop/src/main.tsx	React entry — BrowserRouter, AuthProvider

desktop/src/App.tsx	Route definitions
desktop/src/context/AuthContext.tsx	Auth state, JWT token, billing access
desktop/src/lib/api.ts	Typed HTTP client, JWT headers, error formatting
desktop/src/lib/visualization.ts	Parse arrays, matrices, trees from variable strings
desktop/src/lib/subscription.ts	Client-side trial/subscription helpers
desktop/src/lib/utils.ts	Shared UI utilities
desktop/src/pages/EditorPage.tsx	Split editor + visualization, panel layout, line highlight
desktop/src/pages/MyQuestionsPage.tsx	Question CRUD with Monaco per language
desktop/src/pages/DashboardPage.tsx	Question library grid
desktop/src/pages/LoginPage.tsx	Login form
desktop/src/pages/RegisterPage.tsx	Registration form
desktop/src/pages/BillingPage.tsx	Trial / subscription UI
desktop/src/components/VisualizationPlayer.tsx	Step player, array / matrix / tree renderers
desktop/src/components/ProtectedLayout.tsx	Auth gate, sidebar shell, scroll layout
desktop/src/components/Sidebar.tsx	Navigation and logout
desktop/shared/types.ts	Shared TypeScript interfaces

6.3 Editor + visualization UI

EditorPage provides a resizable split layout:

- **Left:** Monaco editor with language selector
- **Right:** `VisualizationPlayer` with step controls
- Layout modes: focus editor / balanced / focus visualization
- During playback, the current source line is highlighted in Monaco

VisualizationPlayer reads each step's `variables` and renders:

- 1D arrays as bar charts
- 2D arrays as matrix grids
- Java `TreeNode` objects as a binary tree view

7. Backend (Low-Level Design)

7.1 Server core

File	Purpose
desktop/server/index.ts	Express app factory, route mounting, static SPA serving

desktop/server/standalone.ts	Web deployment entrypoint (PORT, bind 0.0.0.0)
desktop/server/db.ts	SQLite schema, admin seed, migrations
desktop/server/auth.ts	JWT sign / verify
desktop/server/middleware.ts	authMiddleware, subscriptionMiddleware
desktop/server/env.ts	Load .env file

7.2 Routes

File	Purpose
desktop/server/routes/auth.ts	Register / login, bcrypt, trial user creation
desktop/server/routes/questions.ts	Owner-scoped CRUD, Zod validation
desktop/server/routes/visualize.ts	Language dispatch, submission logging
desktop/server/routes/billing.ts	Stripe checkout, webhook, portal

7.3 Services

File	Purpose
desktop/server/services/subscription.ts	Trial expiry, access checks, Stripe mapping
desktop/server/services/billing-config.ts	isBillingEnabled() feature flag
desktop/server/services/questions.ts	createQuestionForUser, sample question
desktop/server/services/sampleQuestion.ts	Bubble Sort templates (7 languages)
desktop/server/services/tracer-paths.ts	Resolve tracer files (dev / Docker / packaged)

7.4 Runners

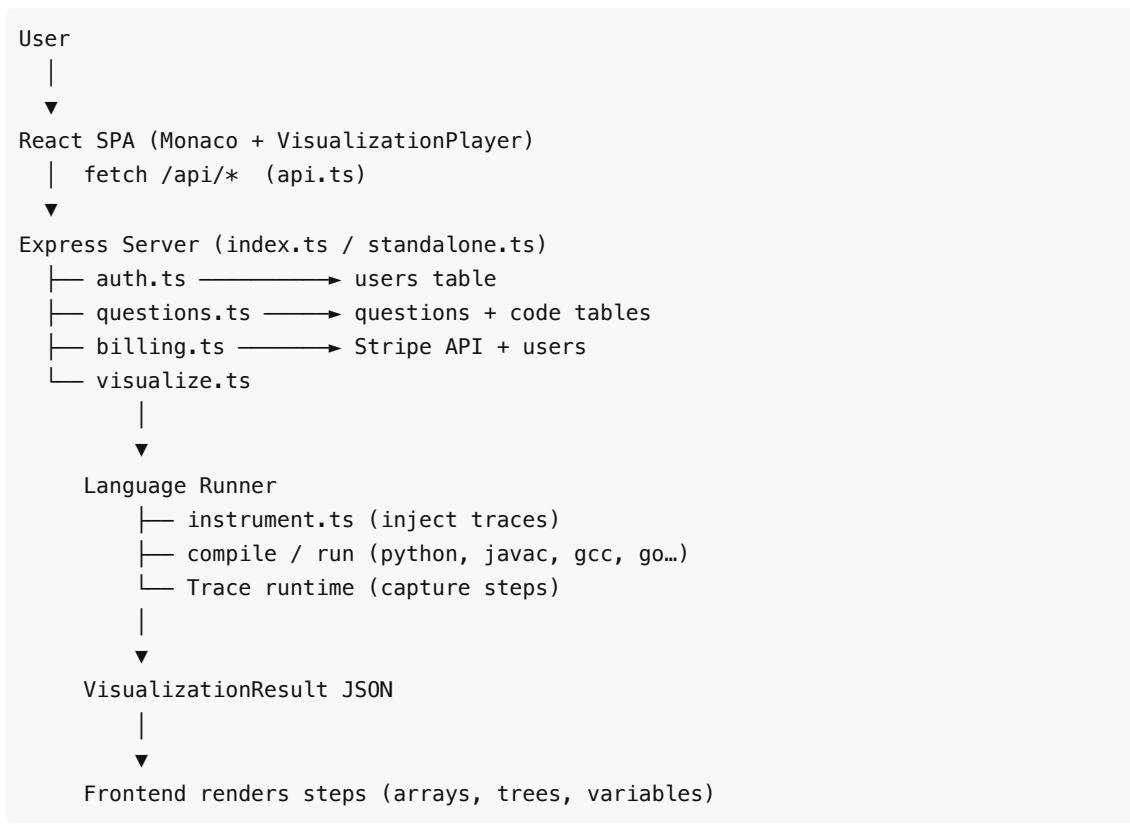
File	Purpose
desktop/server/runners/index.ts	Python (sys.settrace), JavaScript (eval)
desktop/server/runners/java.ts	javac / java compile-run, prepareJavaSource
desktop/server/runners/native.ts	gcc / g++ for C and C++
desktop/server/runners/csharp.ts	C# compile and run
desktop/server/runners/go.ts	go run instrumented source

7.5 Tracers

File	Purpose
desktop/server/java-tracer/instrument.ts	Inject codeviz.Trace.record() calls

desktop/server/java-tracer/Trace.java	Java runtime: steps, TreeNode JSON serialization
desktop/server/native-tracer/instrument.ts	C / C++ line instrumentation
desktop/server/native-tracer/trace.c	C runtime tracer
desktop/server/csharp-tracer/instrument.ts	C# instrumentation
desktop/server/csharp-tracer/Trace.cs	C# runtime tracer
desktop/server/go-tracer/instrument.ts	Go instrumentation
desktop/server/go-tracer/trace.go	Go runtime tracer
desktop/server/instrument/scope.ts	Shared scope tracking for native tracers

7.6 Component interaction



8. Electron, Docker & CI

8.1 Electron

File	Purpose
desktop/electron/main/index.ts	Main process: API server, window, paths
desktop/electron/preload/index.ts	Expose window.codeviz.apiUrl

desktop/scripts/ensure-electron.js	Postinstall Electron binary repair
------------------------------------	------------------------------------

8.2 Docker & Render

File	Purpose
desktop/Dockerfile	Multi-stage: build SPA + runtime with compilers
desktop/.dockerignore	Docker build exclusions
render.yaml	Render Blueprint: codeviz-beta web service
.github/workflows/release.yml	CI: macOS Electron .dmg build on v* tags

8.3 Environment variables

Variable	Purpose
JWT_SECRET	JWT signing key
CODEVIZ_DATA_DIR	SQLite and data directory
CODEVIZ_TRACER_ROOT	Path to tracer source files
BILLING_ENABLED	Enable Stripe billing (default: false)
STRIPE_SECRET_KEY	Stripe API key
STRIPE_PRICE_ID	\$2/month price ID
STRIPE_WEBHOOK_SECRET	Webhook signature verification
PORT	Web server port (default: 10000)

8.4 Local development commands

```
# Desktop app
cd desktop && npm run electron:dev

# Web (local)
cd desktop && npm run build:web && PORT=8080 npm run start:web
```

Demo credentials: admin@codeviz.app / admin123

9. AlgoViz iOS (Separate Product)

AlgoViz/ is a standalone SwiftUI iOS 17+ app with pre-built algorithm simulators (Bubble Sort, Binary Search). It has **no connection** to the CodeViz API or tracers.

File / Folder	Purpose
AlgoViz/AlgoViz/AlgoVizApp.swift	App entry

AlgoViz/AlgoViz/Algorithms/	BubbleSortSimulator, BinarySearchSimulator
AlgoViz/AlgoViz/Models/	Topic, VisualizationStep models
AlgoViz/AlgoViz/ViewModels/	Home and playback state
AlgoViz/AlgoViz/Views/	HomeView, VisualizationDetailView, components
AlgoViz/AlgoViz/Services/TopicCatalog.swift	Static lesson catalog

10. Technology Stack

Layer	Technologies
Frontend	React 19, TypeScript, Vite, Tailwind CSS, Monaco Editor, React Router 7
Backend	Express 5, Node 22, SQLite (node:sqlite), JWT, bcryptjs, Stripe SDK, Zod
Desktop	Electron, electron-builder
Deploy	Docker, Render Blueprint, GitHub Actions
Languages traced	Python, JavaScript, Java, C, C++, C#, Go

End of document